# Elementos básicos de stark

---

**M9:** Fundamentos de big data

|**AE3**: Manipular grandes volúmenes de datos utilizando objetos rdd, transformaciones y acciones para resolver un problema en el entorno spark.

alkemy

# Introducción🧑‍💻

Apache Spark es uno de los motores de procesamiento de datos más potentes y utilizados en la actualidad para manejar grandes volúmenes de información. Su principal característica es la capacidad de realizar análisis masivo de datos de forma distribuida y en memoria, lo que permite obtener resultados de manera más rápida que sistemas tradicionales como Hadoop MapReduce.

Para trabajar con Spark es fundamental conocer sus componentes básicos, entre los que se destacan los **RDD** (Resilient Distributed Datasets), las **transformaciones** y las **acciones**, que permiten manipular datos a gran escala. También es esencial entender cómo funciona la configuración inicial, el contexto de Spark y los conceptos de **linaje** y **evaluación perezosa (lazy evaluation)**, los cuales impactan directamente en la eficiencia del procesamiento.

Este manual te guiará paso a paso por los fundamentos de Spark, desde su configuración inicial hasta la creación de un RDD, aplicando transformaciones y acciones para resolver problemas típicos de procesamiento masivo.

## Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Configurar y conectar un entorno básico de Spark en Python.
- Crear y cargar RDDs a partir de datos distribuidos.
- Aplicar transformaciones como `map`, `filter` y `flatMap` a conjuntos de datos.
- Ejecutar acciones como `collect`, `take` y `sum` para obtener resultados.
- Entender cómo Spark ejecuta Jobs y cómo optimiza los cálculos mediante Lazy Evaluation.

# Configuración, conexión y contexto de Spark

Para trabajar con Spark se debe inicializar un **SparkContext**, que es el punto de entrada para cualquier operación. A través del contexto, Spark administra recursos y coordina la ejecución distribuida.

Ejemplo básico en PySpark:

```
from pyspark import SparkContext
sc = SparkContext("local", "MiAplicacionSpark")
print(sc)
```

- **Configuración local:** `local` indica que el procesamiento se ejecutará en el equipo local.
- **Nombre de la aplicación:** útil para identificar el job en el monitor de Spark.
- **Driver y ejecutores:** Spark coordina tareas mediante un driver y múltiples ejecutores distribuidos.

Tener correctamente configurado el contexto garantiza que las operaciones sobre datos se ejecuten de manera eficiente en clústeres grandes o entornos de prueba locales.

# RDD: Qué es un RDD

Un **RDD (Resilient Distributed Dataset)** es la estructura de datos fundamental de Spark.
Se trata de un conjunto de elementos distribuidos a través de varios nodos de un clúster, que permite:

- **Procesamiento paralelo:** cada partición se procesa de manera independiente.
- **Tolerancia a fallos:** puede reconstruirse a partir de su linaje si un nodo falla.
- **Evaluación diferida:** las operaciones no se ejecutan inmediatamente, sino cuando se requiere un resultado final.

## Creación y carga de un RDD

Podemos crear un RDD de distintas maneras:

1. Desde una colección en Python

```
datos = [1, 2, 3, 4, 5]
rdd = sc.parallelize(datos)
```

2. Desde un archivo de texto

```
rdd = sc.textFile("ruta/archivo.txt")
```

3. Desde una base de datos o fuente externa (mediante conectores adicionales).

## Almacenamiento de un RDD

- **Persistencia en memoria:** `rdd.persist()` mantiene los datos para múltiples usos.
- **Caché:** `rdd.cache()` es equivalente a persistencia en memoria por defecto.

## Pair-RDD

Un **Pair-RDD** es un RDD compuesto por pares clave-valor.
Ejemplo:

```
rdd = sc.parallelize([("a", 1), ("b", 2), ("a", 3)])
```

Este tipo de RDD es fundamental para operaciones de agregación (`reduceByKey`, `groupByKey`) y es ampliamente usado en análisis de grandes volúmenes de datos.

## Linaje y Lazy Evaluation

- **Linaje:** registro de las transformaciones aplicadas a un RDD.

- **Lazy Evaluation:** Spark no ejecuta las transformaciones inmediatamente; espera hasta que se invoque una **acción**.

Este enfoque permite optimizaciones automáticas y evita cálculos innecesarios.

# Transformaciones

Una **transformación** es una operación que crea un nuevo RDD a partir de uno existente.
Son operaciones **perezosas** (lazy), es decir, no ejecutan el cálculo hasta que se invoque una acción.

## Principales métodos

- `filter(func)`: devuelve elementos que cumplen una condición.
- `map(func)`: aplica una función a cada elemento del RDD.
- `flatMap(func)`: igual que `map`, pero devuelve iteradores planos.
- `sample()`: toma una muestra aleatoria del RDD.
- `union()`: une dos RDD en uno solo.
- `distinct()`: elimina duplicados.
- `sortBy()`: ordena el RDD según un criterio.

Ejemplo:

```
rdd = sc.parallelize([1,2,3,4,5])
resultado = rdd.filter(lambda x: x % 2 == 0).map(lambda x: x**2)
print(resultado.collect())  # [4, 16]
```

# Acciones

Una **acción** ejecuta realmente las transformaciones pendientes sobre un RDD y devuelve un resultado.

## Principales acciones

- `collect()`: devuelve todos los elementos como una lista.
- `take(n)`: obtiene los primeros `n` elementos.
- `top(n)`: obtiene los `n` elementos más altos según el orden natural.

- `takeSample()`: extrae una muestra aleatoria.
- `sum()`: suma todos los elementos (numéricos).
- `mean()`: calcula la media de los elementos (requiere `RDD` numérico).
- `stdev()`: calcula la desviación estándar de los datos.

Ejemplo:

```
rdd = sc.parallelize([1,2,3,4,5])
print(rdd.sum())      # 15
print(rdd.take(3))    # [1, 2, 3]
```

# Job Spark: Qué es un Job Spark

Un **Job** en Spark es la unidad de ejecución que se lanza cuando se invoca una **acción**.
El ciclo de un Job incluye:

1. Transformaciones registradas en DAG (Directed Acyclic Graph).
2. División en etapas (stages) según dependencias.
3. Ejecución en tareas distribuidas (tasks) por los ejecutores.

Cuando llamamos a `collect()`, `count()` u otra acción, Spark:

- Evalúa el linaje del RDD.
- Crea un plan de ejecución (DAG).
- Divide el trabajo en stages y tasks.
- Retorna el resultado final al driver.

# Cierre ✍️

El manejo de **RDDs en Spark** constituye la base del procesamiento de datos distribuido.

Comprender cómo crear, almacenar y transformar RDDs, junto con la diferencia entre **transformaciones** y **acciones**, permite construir soluciones escalables y eficientes.

El concepto de **lazy evaluation** y **linaje** ofrece tolerancia a fallos y optimizaciones automáticas, mientras que entender cómo se ejecuta un **Job Spark** brinda claridad sobre el flujo de trabajo interno del motor.

Dominar estos elementos es fundamental para desarrollar aplicaciones que manipulen grandes volúmenes de datos de forma eficiente, cumpliendo los objetivos planteados en entornos de **Big Data**.

¡Nos vemos de nuevo, más adelante! 👋

# Referencias📚


- Apache Spark. (2024). *RDD programming guide*. Apache Spark. https://spark.apache.org/docs/latest/rdd-programming-guide.html
- Databricks. (2023). *Introduction to Apache Spark*. Databricks. https://www.databricks.com/glossary/what-is-apache-spark
- IBM. (2023). *What is RDD in Apache Spark?* IBM Cloud. https://www.ibm.com/cloud/blog/apache-spark-rdd

# ¡Muchas gracias!

Nos vemos en la próxima lección

alkemy